The Journal of Open Source Software

Zarr-Cesium: Browser-native visualization of multidimensional geospatial Zarr data

Tobias Ferreira [1]

1 National Oceanography Centre, United Kingdom

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0)

Summary

Ocean and atmospheric scientists work with datasets that vary across latitude, longitude, depth, and time. Although these data can be stored efficiently as multidimensional arrays, making them explorable on an interactive map often requires a separate service that transforms the data into images. This adds infrastructure and can limit users to a set of predefined views.

`Zarr-Cesium` is an open-source TypeScript library for visualizing cloud-hosted Zarr arrays (Zarr Developers, 2026b) in CesiumJS (Cesium GS, 2026). It reads relevant chunks in the browser and displays scalar fields on a globe, GPU-rendered horizontal and vertical sections through three-dimensional datasets, arbitrary vertical cross-sections, and animated vector fields. Users can change variables, times, depths, and visual styling while working with the underlying multidimensional data rather than a pre-rendered collection of map images. The library supports Zarr v2 and v3, common Climate and Forecast (CF) metadata conventions (Eaton et al., 2025), suitable single-scale and multiscale arrays, and geographic and Web Mercator coordinate reference systems. It is intended for developers building research-oriented, interactive applications, particularly in oceanography, meteorology, and environmental data dissemination.

Statement of need

Environmental simulations and observations frequently describe processes that evolve in several dimensions. Ocean temperature, for example, may change substantially with depth, while ocean currents have distinct horizontal components and temporal variability. Large reanalyses and climate-model archives illustrate both the scale and scientific importance of these data (Eyring et al., 2016; Hersbach et al., 2020). Zarr enables chunked access to selected portions of such arrays without retrieving an entire dataset (Abernathey et al., 2021; Zarr Developers, 2026b). However, efficient storage alone does not provide a way to explore these data in a web-based geographical context.

A common publication workflow serves raster images through a Web Map Service (WMS) or another tile endpoint. These architectures remain useful, especially for centralized processing, access control, or broad client compatibility, but their deployment and maintenance can be disproportionate for projects already distributing suitably structured arrays from object storage. Pre-generating images also introduces derived products that must be managed when datasets or presentation choices change.

`Zarr-Cesium` addresses developers of environmental data portals, research infrastructures, and exploratory scientific tools who need interactive geographic views of multiple dimensions without maintaining a dedicated rasterization service. Its objective is not to replace numerical analysis tools, but to connect cloud-native scientific arrays to interactive 2D and 3D visualization and exploratory queries for points, time series, vertical profiles, and transects.

DRAFT

State of the field

Several established tools cover complementary parts of this workflow. GeoServer and ncWMS provide server-managed map services for geospatial and multidimensional environmental data (Blower et al., 2013; GeoServer, 2026). TiTiler and related xarray-based services generate map tiles from cloud-hosted raster data or multidimensional arrays (Development Seed, 2026c, 2026d; Earthmover, 2026). These are appropriate where server-side rendering, transformation, and centralized control are desired; they do not eliminate the rendering service itself.

Other tools bring the processing closer to the client. Zarrita provides JavaScript access to Zarr arrays (Zarrita Developers, 2026), while CesiumJS supplies a mature geographical rendering engine (Cesium GS, 2026). Neither alone supplies the combined workflow of resolving multidimensional scientific coordinates, selecting array subsets from a geographic view, and rendering those subsets as Cesium imagery and 3D sections. Browser-based Zarr visualization projects also exist, including the CarbonPlan `zarr-layer` and Development Seed `deck.gl-raster`, but target different rendering environments or narrower visualization tasks [CarbonPlan (2026)](Development Seed, 2026b).

We developed `Zarr-Cesium` as an integration layer rather than implementing a new data format, array reader, or 3D globe. It builds on Zarrita, CesiumJS, and reusable Zarr mapping components. The distinctive contribution is the coordinated handling of scientific dimensions, selective retrieval, visualization-specific data preparation, and Cesium rendering through a consistent library interface. This makes it possible to add multidimensional Zarr visualization to an existing Cesium application without first designing and operating a custom tile API.

Software design

Zarr-Cesium separates data access, scientific coordinate interpretation, and CesiumJS rendering. Three public providers offer complementary visualization modes: `ZarrLayerProvider` renders two-dimensional scalar fields as imagery; `ZarrCubeProvider` displays geographic horizontal and vertical sections through three-dimensional scalar arrays; and `ZarrCubeVelocityProvider` animates vector fields using particle advection. Rather than implementing a new array reader or geographic engine, the package combines Zarr-compatible data access with Cesium imagery layers and scene primitives.

The two-dimensional processing pathway shares data selection and tiling logic with Leaflet (Leaflet Developers, 2026) and OpenLayers (OpenLayers Developers, 2026) integrations through `zarr-maps-tiling` (NOC Ocean Informatics, 2026b). The separate `zarr-maps-colormap` package supplies reusable palette definitions and colour-ramp utilities (NOC Ocean Informatics, 2026a). For three-dimensional datasets, both cube providers use the exported, non-rendering `ZarrCubeDataProvider` to resolve dimensions and selectors, select geographic bounds, load a data subset, and normalize coordinate ordering. Separating this loader from scene rendering allows applications to use the selected array and its coordinates without relying on Cesium-specific visualization classes.

The overall architecture is summarised in Figure 1, showing how Zarr-compatible stores and shared data-access components feed the three Cesium-facing visualization providers. This separation allows data retrieval and scientific coordinate handling to be reused independently of individual rendering modes.

DRAFT

The Journal of Open Source Software

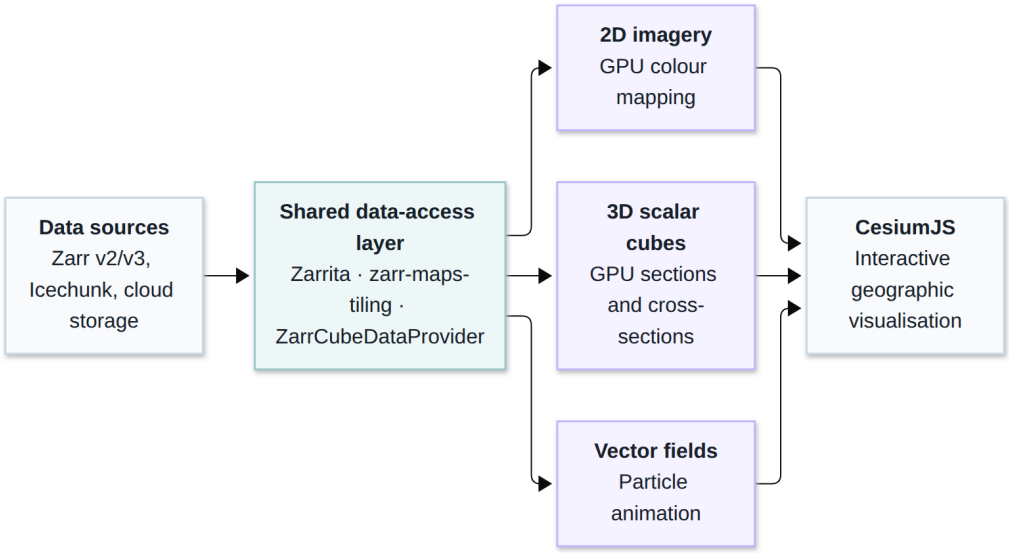


Figure 1: High-level architecture of Zarr-Cesium showing cloud-native data sources, the shared data-access layer and the three visualisation pathways in CesiumJS.

The providers read Zarr v2 and v3 arrays through Zarrita-compatible readable stores (Zarrita Developers, 2026). They can access publicly hosted HTTP datasets, custom readable stores such as Icechunk, and restricted stores through request credentials, per-object header transformations, proxies, or signed URLs. Support for CF-style coordinate metadata (Eaton et al., 2025), configurable dimension names, and coordinate reference systems EPSG:4326 and EPSG:3857 connects array indices to geographic positions. Single-scale data and GeoZarr multiscale layouts (Zarr Developers, 2026a) are supported, although access patterns and performance depend on data layout and the chosen resolution. Scientific query methods expose sampled values, time series, vertical profiles, and transects in addition to the visual layers.

For two-dimensional imagery, WebGL textures and shader-driven colour mapping separate many styling operations from data retrieval. Zarr-Cesium v0.4.0 extends GPU-based rendering to scalar cube sections. Its `CubeGpuRenderer` retains cube values and colour mapping on the GPU and draws horizontal, latitude, and longitude sections using Cesium primitives. It also supports user-defined vertical cross-sections between two geographic coordinates, including projected EPSG:3857 datasets. The renderer updates selected sections, colour scales, opacity, and vertical layout without requiring a new Zarr read when the loaded subset is unchanged. If the cube exceeds the device's supported texture size, `ZarrCubeProvider` uses a CPU-based compatibility renderer that creates RGBA textures. This fallback preserves the principal slice APIs but does not support arbitrary two-point cross-sections. Vector-field animation uses a Cesium wind-layer implementation adapted to three-dimensional cube data (Cesium Wind Layer Developers, 2024).

For tiled scalar imagery, the CPU-to-GPU processing sequence is illustrated in Figure 2. After a tile request, selected Zarr chunks are fetched and decoded on the CPU; numerical values and rendering parameters are then supplied to WebGL, which applies the shader-based colour mapping and produces the image consumed by CesiumJS. This figure describes the two-dimensional tile pathway, not the separate GPU cube renderer.

DRAFT

The Journal of Open Source Software

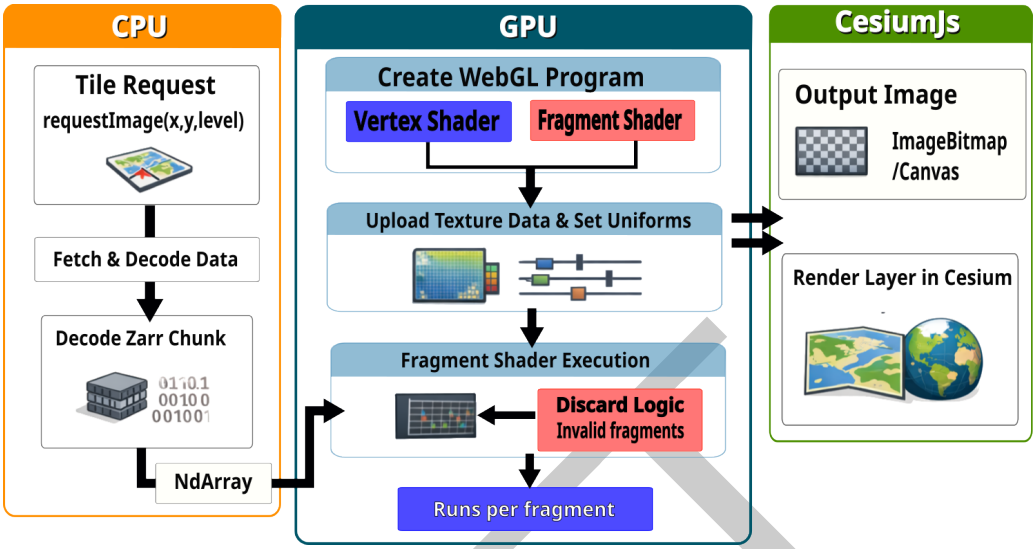


Figure 2: Two-dimensional tile-rendering workflow in Zarr-Cesium: requested Zarr chunks are decoded into numerical arrays on the CPU, transferred to WebGL for shader-based colour mapping, and supplied as imagery to CesiumJS.

These choices move substantial data selection and rendering work into the browser, making it possible to deploy a scientific viewer as static application assets alongside an accessible Zarr store without operating a dedicated raster tile server. They also impose constraints: source data must be chunked appropriately, browser memory and GPU texture limits bound the size of interactive cubes, HTTP access must permit cross-origin requests, and very large datasets may benefit from server-assisted processing. GPU-rendered cube slices are not equivalent to full volumetric ray casting; they provide interactive sections through a loaded subset.

Research impact statement

`Zarr-Cesium` was developed for the AtlantiS environmental visualization platform at the National Oceanography Centre. In that application, it provides browser-based views of multidimensional oceanographic and related environmental datasets in a geographic context (National Oceanography Centre, 2026). This constitutes an application within a research infrastructure project, rather than evidence of independent adoption.

Independent integration is documented in GeoLibre, an open-source geospatial application maintained outside the original development team. GeoLibre declares `zarr-cesium` as a dependency and uses `ZarrLayerProvider` to display Zarr data on its Cesium globe (Wu & GeoLibre Contributors, 2026). Its maintenance notes describe the integration, compatibility tests, and limitations concerning colormap options and dimension aliases (GeoLibre Contributors, 2026). These details offer evidence that a downstream developer has exercised the library's interfaces and identify concrete opportunities for upstream improvements. They do not, on their own, establish a peer-reviewed scientific result produced with the software.

The library is also described in Development Seed's *Datacube Guide* alongside other approaches to visualizing cloud-native arrays (Development Seed, 2026a). This independent technical documentation demonstrates recognition and discoverability, but is not counted here as a separate scientific deployment. Together, AtlantiS and GeoLibre demonstrate research-oriented application and external software uptake.

DRAFT

AI usage disclosure

Generative AI tools, including OpenAI ChatGPT (GPT-5.6 and GPT-6), were used to assist with the organisation, editing, and refinement of this manuscript, as well as the development and integration of GPU-based rendering for three-dimensional cube visualisation. All software architecture decisions, technical implementations, and scientific claims were reviewed and validated by the author. Any AI-assisted code was manually reviewed, tested, and verified before being incorporated into the software.

Acknowledgements

The author acknowledges the CesiumJS, Zarrita, and related open-source communities. The project was developed in the context of the National Oceanography Centre's AtlantiS initiative. JASMIN provided storage and testing infrastructure.

References

Abernathey, R., Augspurger, T., Banihirwe, A., Blackmon-Luca, C. C., Crone, T., Gentemann, C., Hamman, J., Henderson, N., Lepore, C., McCaie, T., Robinson, N., & Signell, R. P. (2021). Cloud-native repositories for big scientific data. *Computing in Science & Engineering*, *23*(2), 26–35. https://doi.org/10.1109/MCSE.2021.3059437

Blower, J. D., Gemmell, A. L., Griffiths, G. H., Haines, K., Santokhee, A., & Yang, X. (2013). A web map service implementation for the visualization of multidimensional gridded environmental data. *Environmental Modelling & Software*, *47*, 218–224. https://doi.org/10.1016/j.envsoft.2013.04.002

CarbonPlan. (2026). *zarr-layer: Render multidimensional Zarr data in the browser*. https://github.com/carbonplan/zarr-layer

Cesium GS. (2026). *CesiumJS*. https://cesium.com/platform/cesiumjs/

Cesium Wind Layer Developers. (2024). *cesium-wind-layer*. https://github.com/hongfaqiu/cesium-wind-layer

Development Seed. (2026a). *Datacube Guide: Zarr-Cesium*. https://developmentseed.org/datacube-guide/latest/visualization/zarr-cesium.html

Development Seed. (2026b). *deck.gl-raster: Browser-based raster visualization*. https://github.com/developmentseed/deck.gl-raster

Development Seed. (2026c). *TiTiler: A modern dynamic tile server for raster data*. https://github.com/developmentseed/titiler

Development Seed. (2026d). *titiler-xarray*. https://github.com/developmentseed/titiler-xarray

Earthmover. (2026). *xpublish-tiles*. https://github.com/earth-mover/xpublish-tiles

Eaton, B., Gregory, J., Drach, B., Taylor, K., Hankin, S., Caron, J., Signell, R., Bentley, P., Rappa, G., Höck, H., Pamment, A., Juckes, M., Raspaud, M., Blower, J., Horne, R., Whiteaker, T., Blodgett, D., Zender, C., Lee, D., … Laake, P. V. (2025). *NetCDF Climate and Forecast (CF) Metadata Conventions* (Version 1.13). CF Community. https://doi.org/10.5281/zenodo.17801666

Eyring, V., Bony, S., Meehl, G. A., Senior, C. A., Stevens, B., Stouffer, R. J., & Taylor, K. E. (2016). Overview of the coupled model intercomparison project phase 6 (CMIP6) experimental design and organization. *Geoscientific Model Development*, *9*, 1937–1958. https://doi.org/10.5194/gmd-9-1937-2016

DRAFT

GeoLibre Contributors. (2026). *GeoLibre maintenance notes: zarr-cesium integration*. https://geolibre.app/maintenance/

GeoServer. (2026). *GeoServer*. https://geoserver.org/

Hersbach, H., Bell, B., Berrisford, P., Hirahara, S., Horanyi, A., Munoz-Sabater, J., Nicolas, J., Peubey, C., Radu, R., Schepers, D., & others. (2020). The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, *146*(730), 1999–2049. https://doi.org/10.1002/qj.3803

Leaflet Developers. (2026). *Leaflet: An open-source JavaScript library for mobile-friendly interactive maps*. https://leafletjs.com/

National Oceanography Centre. (2026). *AtlantiS environmental data visualisation tool*. https://app.atlantis.ac.uk/

NOC Ocean Informatics. (2026a). *zarr-maps-colormap: Shared colormap utilities for browser-based Zarr visualization*. https://github.com/NOC-OI/zarr-maps

NOC Ocean Informatics. (2026b). *zarr-maps-tiling: Shared multidimensional Zarr mapping components*. https://github.com/NOC-OI/zarr-maps

OpenLayers Developers. (2026). *OpenLayers: A JavaScript library for displaying maps*. https://openlayers.org/

Wu, Q., & GeoLibre Contributors. (2026). *GeoLibre: Open-source geospatial application*. https://github.com/opengeos/GeoLibre

Zarr Developers. (2026a). *GeoZarr specification*. https://github.com/zarr-developers/geozarr-spec

Zarr Developers. (2026b). *Zarr: Open standard for chunked, compressed n-dimensional arrays*. https://zarr.dev/

Zarrita Developers. (2026). *Zarrita: A minimal and modular Zarr implementation in TypeScript*. https://github.com/manzt/zarrita.js

DRAFT